

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration: 1 Hour
Total Marks: 50

Annexure A

Scenario Based Assignment

Code of Conduct:

(192511415)

I Kanish Kumar S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Kanish Kumar

CO2

ASSESSMENT TOOL - 2

SCENARIO:

An airport is designing an AI system to automatically assign arriving flights to available gates. The complexity increases as the number of flights and gates grows.

TASKS:

Formulate the problem as a Constraint Satisfaction Problem (CSP).

(i) Variables, Domains and Constraints,

A Constraint Satisfaction Problem (CSP) consists of Variables, domains and Constraints.

Variables: Each arriving flight is a Variable

$$F_1, F_2, F_3, \dots, F_n.$$

Domains: Each flight can be assigned to one of the available gates:

$$D(F_i) = \{G_1, G_2, \dots, G_m\}$$

Constraints:

- Two flights with overlapping times cannot use the same gate.
- Wide-body aircraft can use only wide-body gates.
- Airline gate preferences should be considered.
- Ground crew must be available for the assigned flight.
- Unavailable or closed gates cannot be selected.

ii) BACKTRACKING SEARCH:

Backtracking assigns gates to flights one by one.

1. Select an unassigned flight.
2. Choose a gate from its domain.
3. Check whether the assignment satisfies all constraints.
4. If valid, assign it and continue.
5. If invalid, try another gate.
6. If no gate is possible, go back and change the previous assignment.

iii) CONSTRAINT PROPAGATION - FORWARD CHECKING:

Forward checking removes impossible gate values from the domains of future flights.

For example, if:

$F_1 = G_1$ and F_2 overlaps with F_1 then G_1 is removed from F_2 's domain.

This reduces the search space and detects conflicts earlier, making backtracking faster.

iv) REAL-WORLD CHALLENGES SCALABILITY:

Real airports have many flights, delays, gate failures, staff limitations and changing schedules.

A good strategy is:

Backtracking + Forward checking + MRV

MRV (Minimum Remaining Values) selects the flight having the fewest available gates first. This improves efficiency and makes the system more scalable.

SCENARIO:

board - game company is developing an AI opponent for a turn-based strategy game similar to checkers, to be deployed on low-power tablets.

TASKS:

(i) MINIMAX ALGORITHM:

Minimax is suitable for two-player turn-based games.

* MAX: AI player tries to maximize the score.

* MIN: Opponent tries to minimize the score.

The game is represented as a search tree.

MAX = max (child)

MIN = min (child)

The AI chooses the move that gives the best outcome assuming the opponent also plays intelligently.

(ii) COMPUTATIONAL CHALLENGES:

Games trees can become extremely large because every position may have many possible moves.

If b is the branching factor and d is the search depth:

$$\text{Time} = O(b^d)$$

Therefore, searching the entire tree requires a lot of CPU time and memory, which is difficult on Low power tablets.

(iii) ALPHA - BETA PRUNING:

Alpha - Beta Pruning improves Minimax by eliminating branches that cannot affect the final answer.

* α = Best Value currently available to MAX.
* β = Best Value currently available to MIN.
When,

$$\alpha \geq \beta$$

The remaining branches can be ignored.

Therefore, Alpha-Beta allows the AI to search deeper without examining every node.

(iv) EVALUATION FUNCTION:

When the complete game cannot be searched, an evaluation function estimates the board position.

A simple function is:

$$\text{Eval} = \text{Piece Advantage} + \text{Mobility} + \text{Position}$$

It considers the number / value of pieces, available moves, strategic positions and threats.

(v) BEST STRATEGY:

For a low-power Tablet, use:

Alpha-Beta Pruning + Iterative Deepening

Iterative Deepening Searches depth 1, then 2, then 3 and so on until the time limit. This gives a strong move while maintaining quick response time.

SCENARIO:

A city's emergency dispatch center uses an AI routing system to send ambulance to accident sites during peak-hour traffic.

TASKS:

(i) GREEDY BEST FIRST SEARCH:

Greedy Best-First Search selects the node with the smallest heuristic.

$h(n)$

Here,

$h(n)$ can represent the straight-line distance to the accident search location.

ADVANTAGES:

- * Fast
- * Simple
- * Useful when a quick decision is needed.

DISADVANTAGES:

- * Ignores the cost already travelled.
- * May select congested or unsafe roads.
- * Does not guarantee the minimum-cost route.

(ii) A* SEARCH:

A* uses both actual cost and heuristic:

$$f(n) = g(n) + h(n)$$

Where,

$g(n)$ → actual travel cost from the starting point.

$h(n)$ → estimated cost to the destination.

$f(n)$ → estimated total cost.

(iii) EFFECT OF HEURISTIC ACCURACY:

A good heuristic helps both algorithms search faster. Greedy Best-first search depends strongly on $h(n)$.

So an inaccurate heuristic can lead to a poor route.

A^* is more reliable because it considers both;

$$g(n) + h(n)$$

For example, a road may have a small geographical distance but very high traffic. A^* can consider the actual travel cost.

(iv) DYNAMIC CHANGES:

Road conditions may suddenly change because of accidents, construction or traffic.

A previously calculated route may therefore become unsuitable.

The system should monitor real-time updates and re-plan the route when major changes occur.

(v) RECOMMENDED ALGORITHM:

The best choice is,

A^* with Dynamic Replanning

A^* provides a good balance between speed, route quality, travel cost and safety. Dynamic replanning allows the ambulance to respond to sudden road closures and traffic changes.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

$$20 \times 20 + 20 \times 20 + 25 \times 18 + 12$$

$$= 92$$